



Department of Computer Science & Engineering

Project Report



Submitted By:

Jannatul Ferdous Tania (ID:C241486)

Submitted To:

Mrs. Israt Binteh Habib
Lecturer, Dept. of CSE

COURSE CODE: CSE-3528
COURSE TITLE: Compiler Lab
SEMESTER: 5th
SECTION: 5CF

A Mini C++ Compiler

Introduction:

A compiler is a software program that translates a high-level programming language into machine-level instructions. The compiler performs multiple phases such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and final code generation. The purpose of this project is to develop a simplified C++ compiler that demonstrates the working principles of a real compiler. To make learning more interactive, the compiler is designed using an Animal Kingdom theme where each compiler phase is represented by a different animal and users collect magic crystals after successfully completing each phase. The compiler accepts source code from the user, analyzes it, generates intermediate code, optimizes the code, produces assembly-like instructions, and finally generates a complete report.

Objective:

- ❑ Compiler Phase Implementation: Hands-on execution of all core compiler phases from lexical analysis to final assembly code generation.
- ❑ Gamified Learning: Simplifying complex compiler design concepts into an interactive animal-themed adventure with 6 magic crystals.
- ❑ Error Detection & Reporting: Detecting lexical, syntax, and semantic errors while generating a detailed symbol table, optimized TAC, and text report.

Methodology:

- ➔ The project follows these compiler phases
- ➔ User inputs C++ source code.
- ➔ Lexical Analysis (Token generation)
- ➔ Symbol Table Creation
- ➔ Syntax Analysis
- ➔ Semantic Analysis
- ➔ Intermediate Code Generation (Three Address Code)
- ➔ Code Optimization
- ➔ Final Assembly Code Generation
- ➔ Report Generation

Literature Review:

While developing this project, the following compiler concepts were studied:

- ❑ **Lexical Analysis** – Reads the source code and breaks it into tokens. (*Ant*)
- ❑ **Tokens** – The smallest parts of code, like int, x, 10, +.
- ❑ **Keywords** – Reserved words of the language, like int, if, while. They cannot be used as variable names.

- ❑ **Identifiers** – User-defined names, like age, sum, main.
- ❑ **Symbol Table** – Stores information about identifiers, such as name, type, value, and scope.
- ❑ **Syntax Analysis** – Checks whether the code follows grammar rules (e.g., correct ; and matching {}).
(Owl)
- ❑ **Semantic Analysis** – Checks the meaning of the code (e.g., variable declared before use, correct data types). (Eagle)
- ❑ **Intermediate Code Generation** – Converts the source code into an intermediate form. (Bee)
- ❑ **Three Address Code (TAC)** – A simple intermediate code format, e.g., $t1 = a + b$.
- ❑ **Code Optimization** – Makes the code faster and removes unnecessary parts without changing the output. (Fox)
- ❑ **Assembly Code Generation** – Converts the optimized code into assembly instructions like LOAD, ADD, and STORE. (Lion)

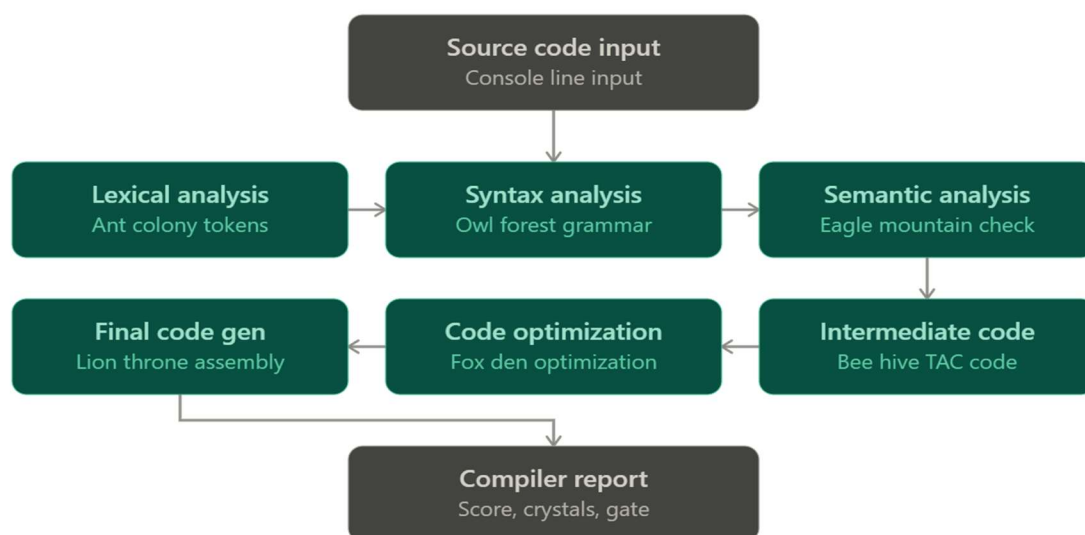


Figure : Overall System Architecture of the Animal Kingdom Mini C++ Compiler

Books Used:

Compilers: Principles, Techniques and Tools

- Alfred V. Aho
- Monica S. Lam
- Ravi Sethi
- Jeffrey D. Ullman

Compiler Design

- Kenneth C. Loudon

Modern Compiler Implementation in C

- Andrew W. Appel

Implementation:

The project is implemented using C++.

Features

- ★ Input Source Code
- ★ Display Source Code
- ★ Lexical Analysis
- ★ Symbol Table Generation
- ★ Syntax Analysis
- ★ Semantic Analysis
- ★ Three Address Code Generation
- ★ Code Optimization
- ★ Assembly Code Generation
- ★ Error Summary
- ★ Statistics
- ★ Reward System
- ★ Text Report Generation
- ★ Auto Compiler

Tools, Language & Software:

Category	Details / Tools
Programming Language	C++
IDE	CodeBlocks
Compiler	GNU GCC (g++)
Operating System	Windows 11
Libraries Used	iostream, vector, string, stream, algorithm, map, set, fstream, windows.h, iomanip, cctype

Hardware

- Processor: Intel Core i5
- RAM: Minimum 4 GB

Software

- Windows 11
- Code::Blocks

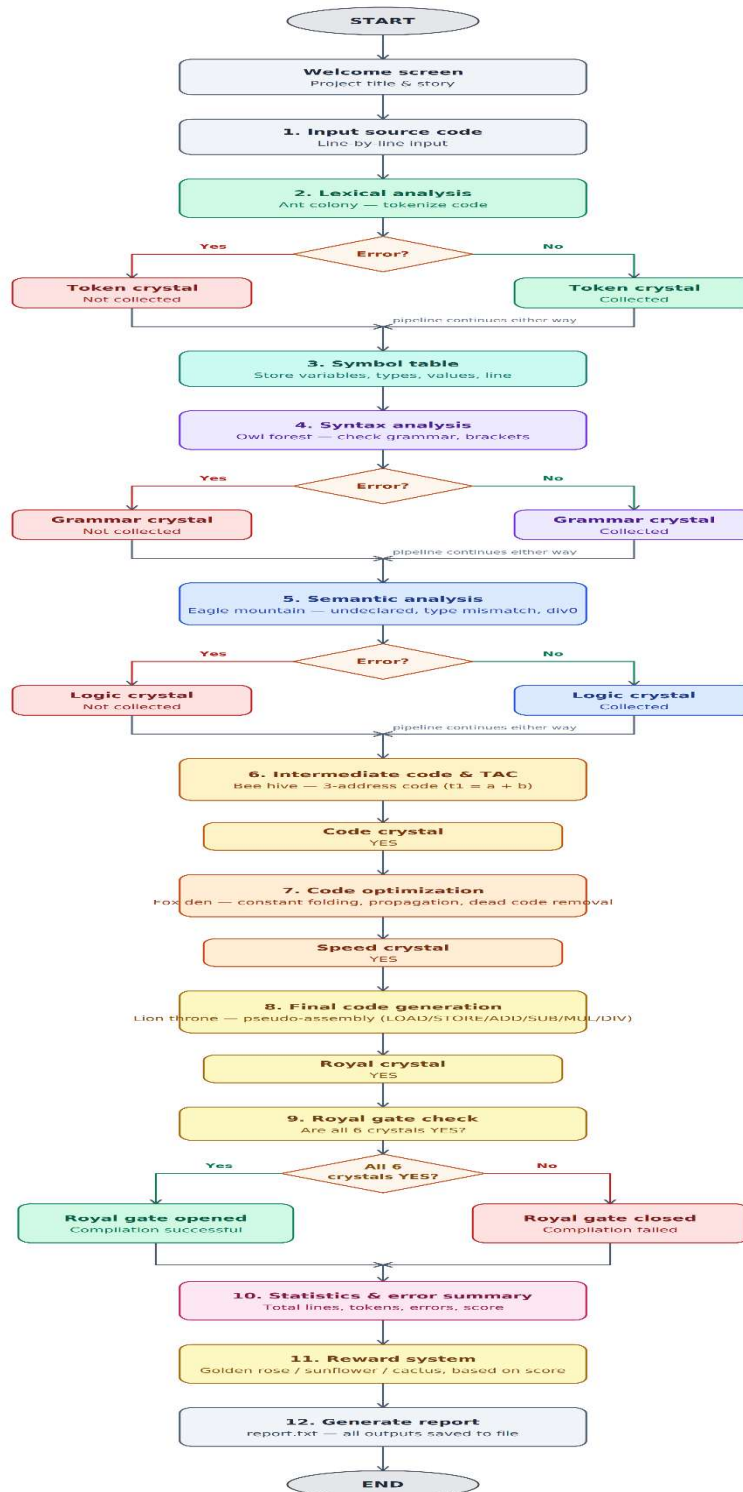
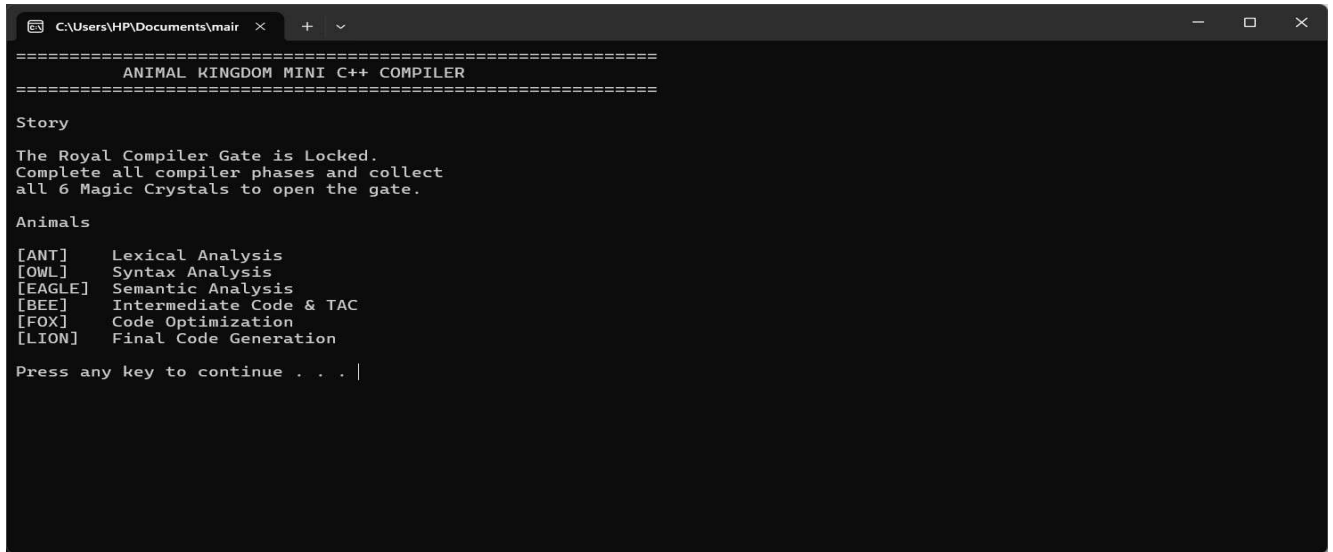


Figure : Flow Graph of Animal Kingdom Mini C++ Compiler

Result:



```
=====
ANIMAL KINGDOM MINI C++ COMPILER
=====

Story

The Royal Compiler Gate is Locked.
Complete all compiler phases and collect
all 6 Magic Crystals to open the gate.

Animals

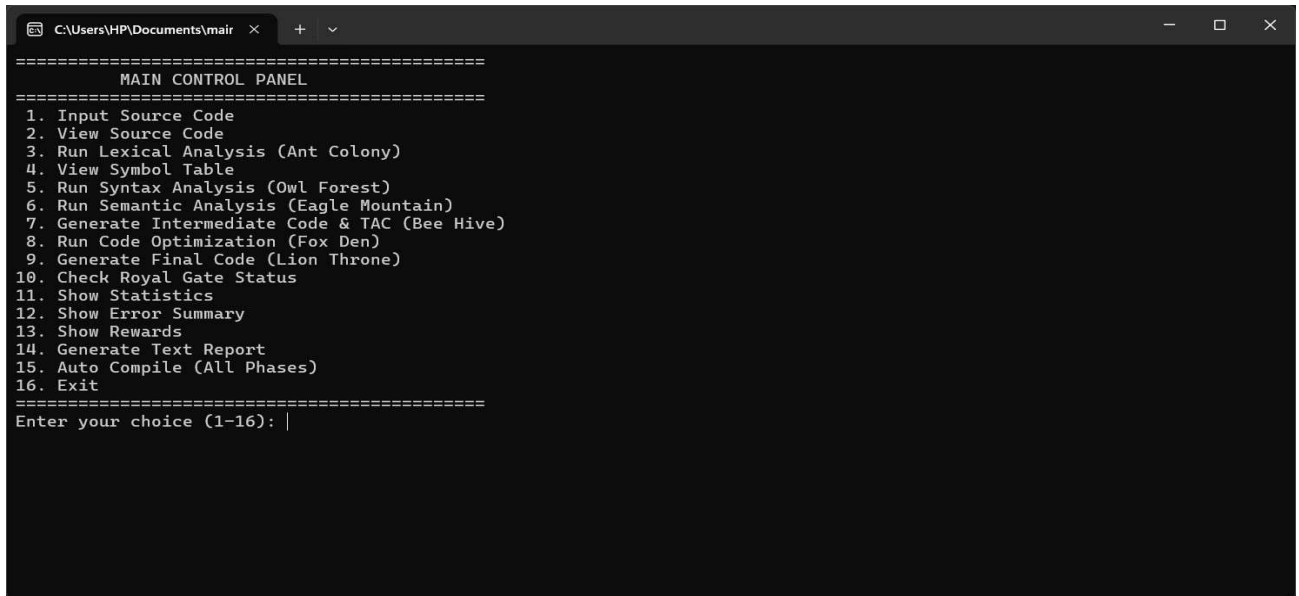
[ANT]   Lexical Analysis
[OWL]   Syntax Analysis
[EAGLE] Semantic Analysis
[BEE]   Intermediate Code & TAC
[FOX]   Code Optimization
[LION]  Final Code Generation

Press any key to continue . . . |
```

Figure :Animal kingdom Mini C++ compiler-Welcome screen

1. Welcome Screen

- Displays a story and lists the six animals (ANT, OWL, etc.) representing each phase.
- Press any key to enter the main menu.



```
=====
MAIN CONTROL PANEL
=====

1. Input Source Code
2. View Source Code
3. Run Lexical Analysis (Ant Colony)
4. View Symbol Table
5. Run Syntax Analysis (Owl Forest)
6. Run Semantic Analysis (Eagle Mountain)
7. Generate Intermediate Code & TAC (Bee Hive)
8. Run Code Optimization (Fox Den)
9. Generate Final Code (Lion Throne)
10. Check Royal Gate Status
11. Show Statistics
12. Show Error Summary
13. Show Rewards
14. Generate Text Report
15. Auto Compile (All Phases)
16. Exit

Enter your choice (1-16): |
```

Figure: Main Control panel of the Compiler

2. Main Menu

Shows 16 options. we can choose phases individually or use Auto Compile (15) to run all phases in sequence.

```
C:\Users\HP\Documents\mair x + v - □ x
=====
YOUR SOURCE CODE
=====

1. int x = 4 + 4;
2. float y = 4;
3. int z = x + y;

Press any key to continue . . . |
```

Figure :Source Code Input/view source code screen

3.& 4. Source Code Input (Option 1), View Source Code (Option 2)

- Asks for the number of lines, then lets you type each line.
- Output: “*Saving Program... Done!*”.
- Prints the Stored Code With Line Numbers

```
C:\Users\HP\Documents\mair x + v - □ x
=====
ANT COLONY - LEXICAL ANALYSIS
=====

Collecting Tokens..... Done!
TOKEN                                TYPE
-----
int                                  Keyword
x                                    Identifier
=                                    Operator
4                                    Number
+                                    Operator
4                                    Number
;                                    Separator
float                                Keyword
y                                    Identifier
=                                    Operator
4                                    Number
;                                    Separator
int                                  Keyword
z                                    Identifier
=                                    Operator
x                                    Identifier
+                                    Operator
y                                    Identifier
;                                    Separator

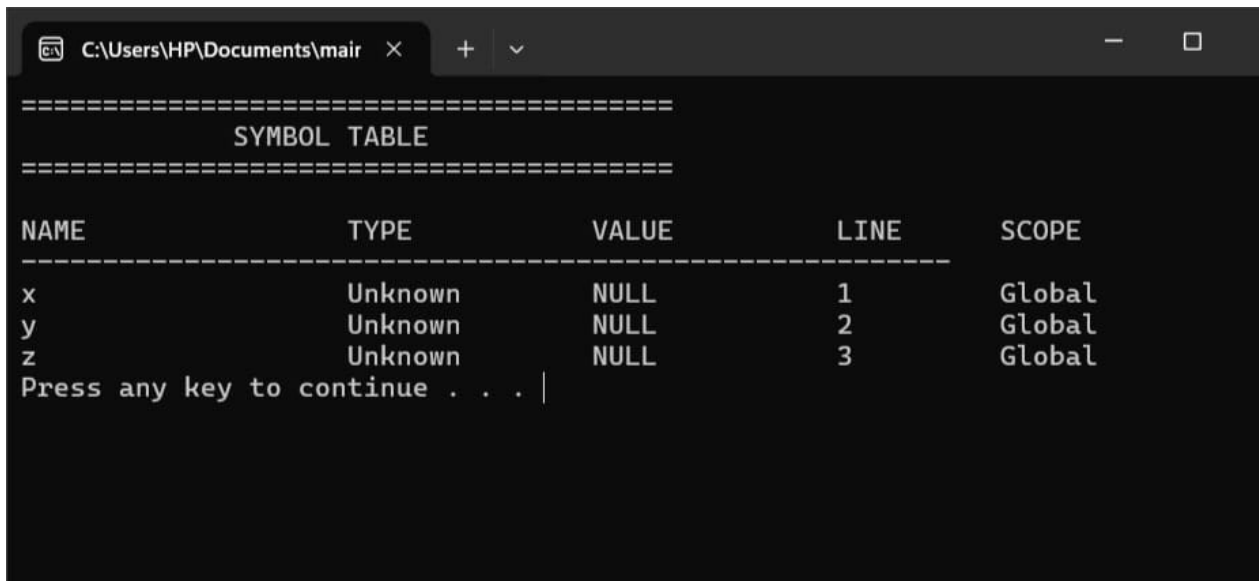
Total Tokens : 19
Keywords: 3, Identifiers: 5, Numbers: 3, Operators: 5, Separators: 3

[SUCCESS] Token Crystal Collected
Press any key to continue . . . |
```

Figure : Lexical Analysis Output(Token Generation)

5. Lexical Analysis – “Ant Colony” (Option 3)

- Scans each line, splits into tokens, and prints a table: TOKEN and TYPE (Keyword, Identifier, Number, Operator, Separator, Invalid).
- Counts total tokens and per-category counts.
- Collects identifiers and builds the Symbol Table (name, type, value, line, scope).
- If invalid tokens exist, prints error count, lowers score, and the Token Crystal is lost.
- Otherwise: “*Token Crystal Collected*”.



The screenshot shows a terminal window with a dark background. At the top, the window title is "C:\Users\HP\Documents\mair". The output text is as follows:

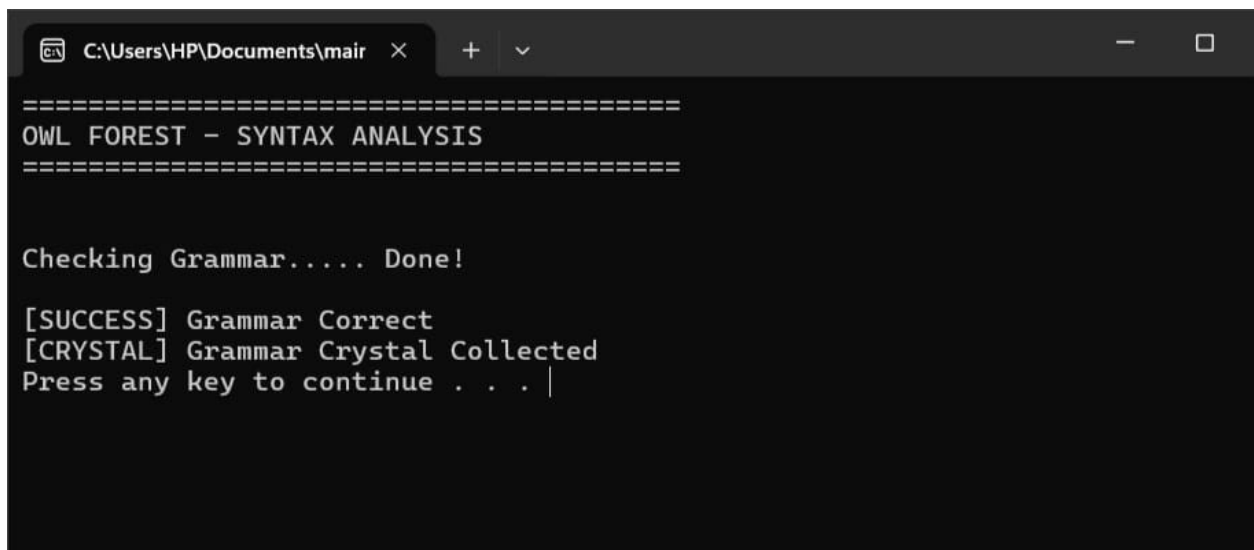
```
=====
                        SYMBOL TABLE
=====

NAME                TYPE        VALUE        LINE        SCOPE
-----
x                   Unknown    NULL         1           Global
y                   Unknown    NULL         2           Global
z                   Unknown    NULL         3           Global
Press any key to continue . . . |
```

Figure :Symbol Table Before Semantic Analysis

6. View Symbol Table (Option 4)

- Prints a formatted table of all symbols found (name, type, value, line, scope).



The screenshot shows a terminal window with a dark background. At the top, the window title is "C:\Users\HP\Documents\mair". The output text is as follows:

```
=====
OWL FOREST - SYNTAX ANALYSIS
=====

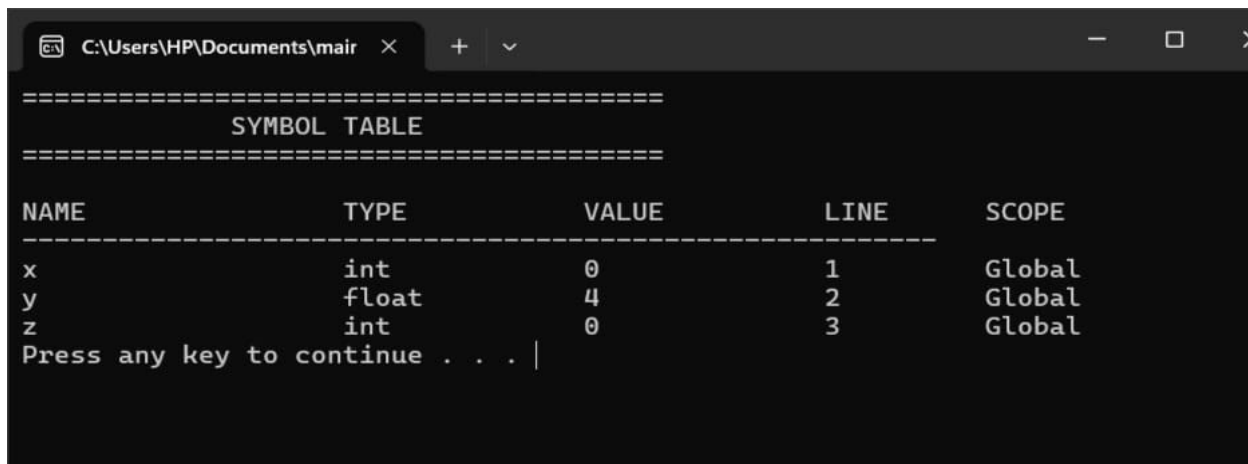
Checking Grammar..... Done!

[SUCCESS] Grammar Correct
[CRYSTAL] Grammar Crystal Collected
Press any key to continue . . . |
```

Figure :Syntax Analysis Output

7. Syntax Analysis – “Owl Forest” (Option 5)

- Checks for:
 - Missing semicolons (;) at line ends.
 - Mismatched parentheses (), braces {}, brackets [].
 - Unclosed string or character literals.
 - Invalid operator ===.
- Prints each error with line number.
- If no errors: “*Grammar Crystal Collected*”. Otherwise, the Grammar Crystal is lost and score decreases.



```
=====
                        SYMBOL TABLE
=====
```

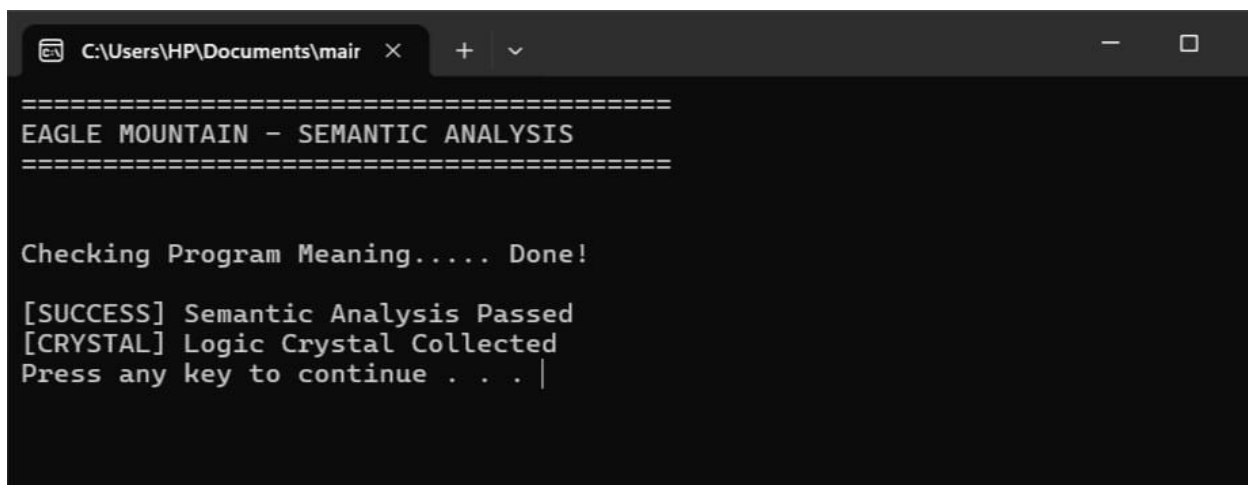
NAME	TYPE	VALUE	LINE	SCOPE
x	int	0	1	Global
y	float	4	2	Global
z	int	0	3	Global

Press any key to continue . . . |

Figure :Symbol Table after Semantic Analysis

6. View Symbol Table (Option 4)

- Prints a formatted table of all symbols found (name, type, value, line, scope).



```
=====
EAGLE MOUNTAIN - SEMANTIC ANALYSIS
=====

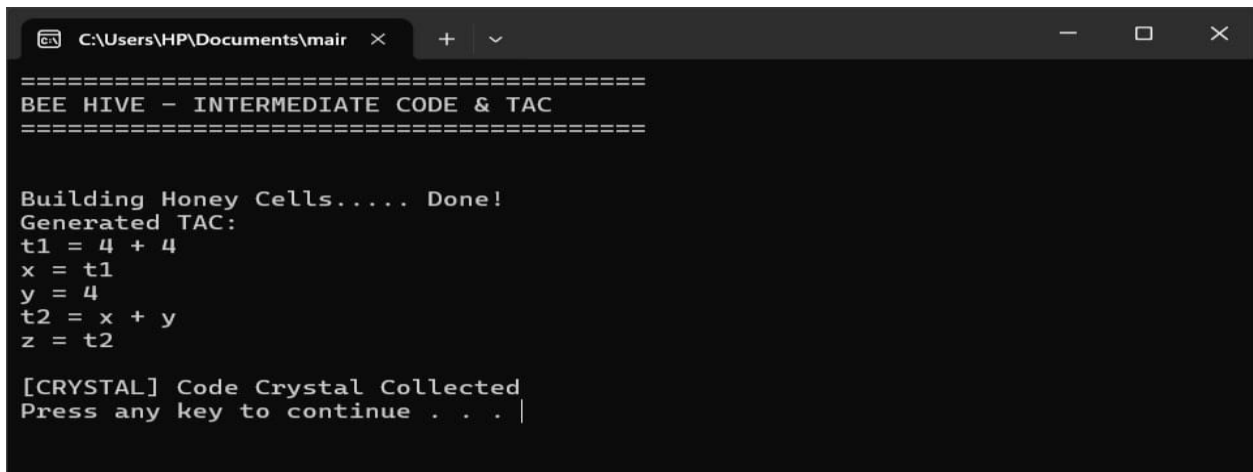
Checking Program Meaning..... Done!

[SUCCESS] Semantic Analysis Passed
[CRYSTAL] Logic Crystal Collected
Press any key to continue . . . |
```

Figure : Semantic analysis

8. Semantic Analysis – “Eagle Mountain” (Option 6)

- Tracks declared variables and their data types.
- Detects undeclared variables (used before declaration).
- Updates the symbol table with types and initial values.
- Prints errors like "Undeclared variable -> x".
- If no errors: “*Logic Crystal Collected*”. Otherwise, the Logic Crystal is lost.



```
C:\Users\HP\Documents\mair x + v - □ X
=====
BEE HIVE - INTERMEDIATE CODE & TAC
=====

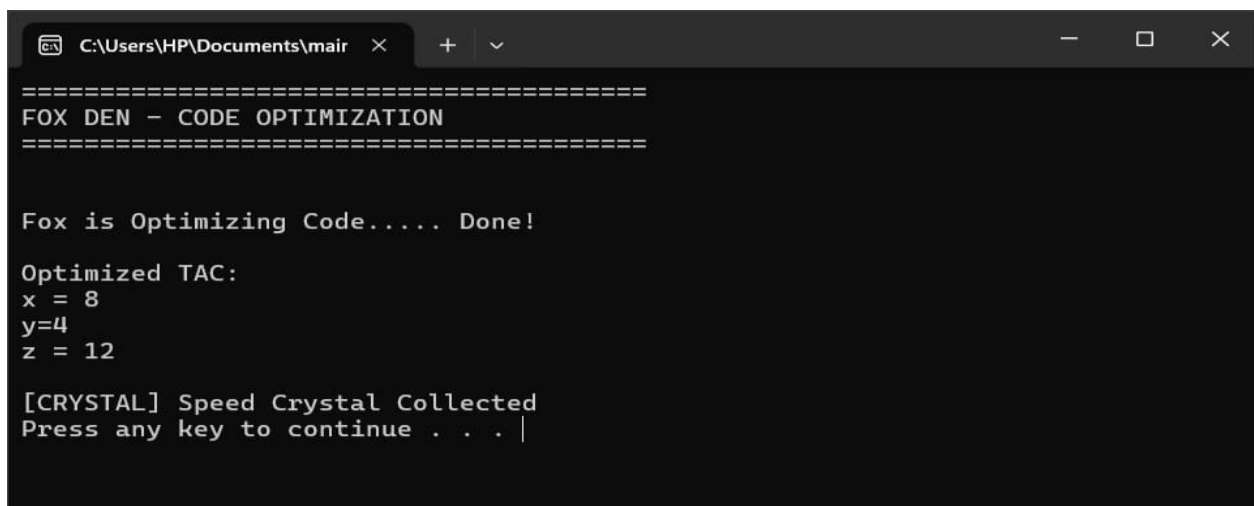
Building Honey Cells..... Done!
Generated TAC:
t1 = 4 + 4
x = t1
y = 4
t2 = x + y
z = t2

[CRYSTAL] Code Crystal Collected
Press any key to continue . . . |
```

Figure :Intermediate code Generation & TAC

9. Intermediate Code & TAC – “Bee Hive” (Option 7)

- Converts each assignment statement into three-address code (TAC) using temporary variables (t1, t2, ...).
- Prints the generated TAC instructions.
- Always collects the Code Crystal.



```
C:\Users\HP\Documents\mair x + v - □ X
=====
FOX DEN - CODE OPTIMIZATION
=====

Fox is Optimizing Code..... Done!

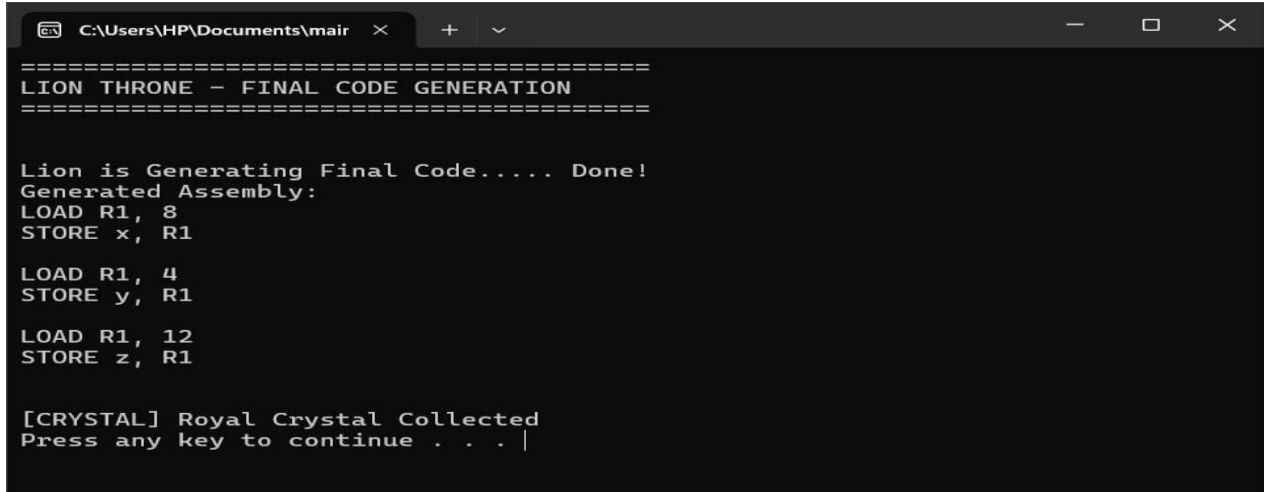
Optimized TAC:
x = 8
y=4
z = 12

[CRYSTAL] Speed Crystal Collected
Press any key to continue . . . |
```

Figure :Code Optimization output

10. Code Optimization – “Fox Den” (Option 8)

- Performs constant propagation and folding (e.g., $t1 = 2 + 3 \rightarrow t1 = 5$).
- Removes redundant temporary variables and unused assignments.
- Prints the optimized TAC.
- Collects the Speed Crystal.



```
=====
LION THRONE - FINAL CODE GENERATION
=====

Lion is Generating Final Code..... Done!
Generated Assembly:
LOAD R1, 8
STORE x, R1

LOAD R1, 4
STORE y, R1

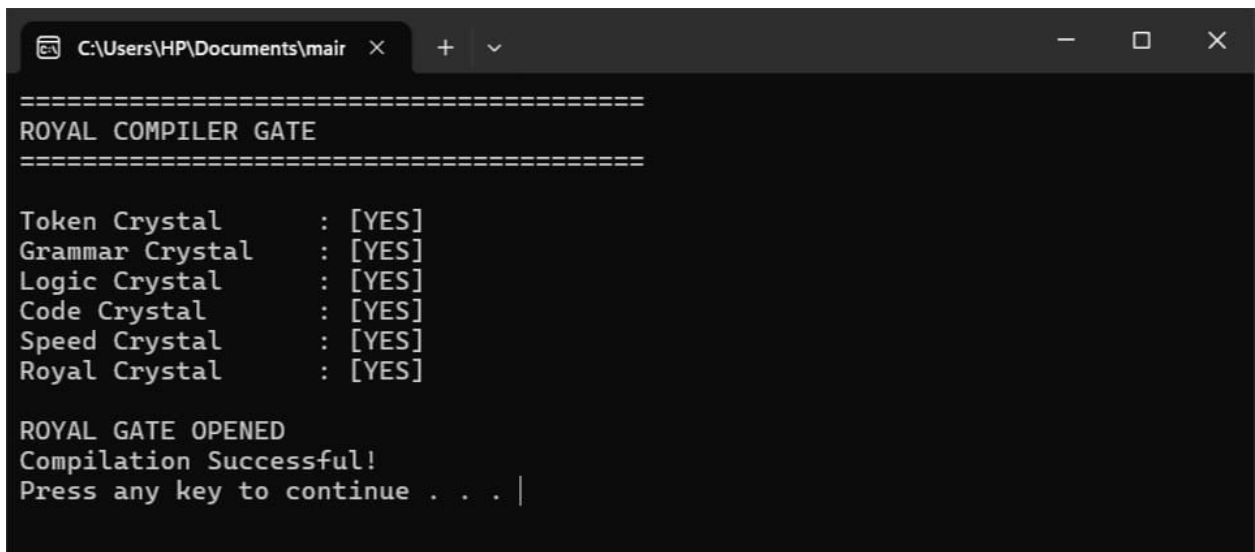
LOAD R1, 12
STORE z, R1

[CRYSTAL] Royal Crystal Collected
Press any key to continue . . . |
```

Figure :Final Code Generation

11. Final Code Generation – “Lion Throne” (Option 9)

- Translates optimized TAC into a simple assembly-like code (LOAD, ADD, SUB, STORE, etc.).
- Prints the generated assembly instructions.
- Collects the Royal Crystal.



```
=====
ROYAL COMPILER GATE
=====

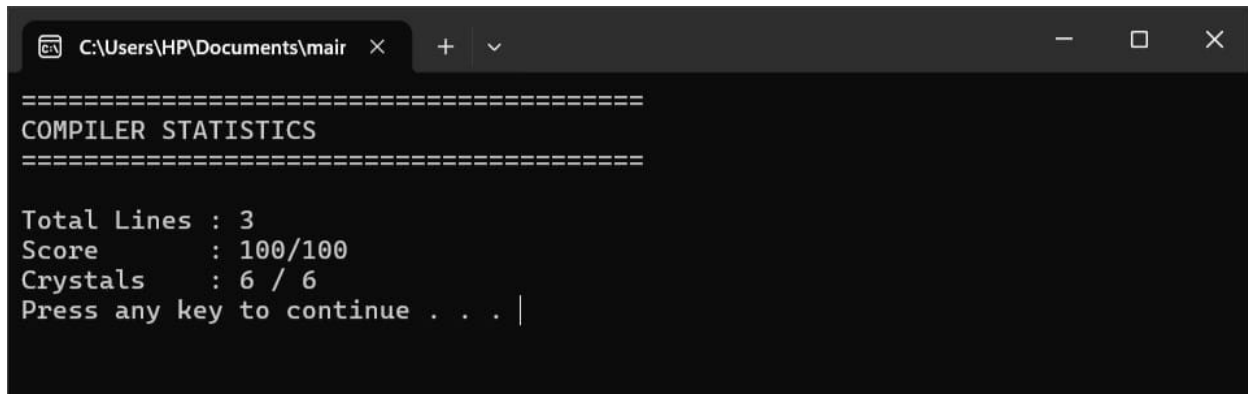
Token Crystal      : [YES]
Grammar Crystal    : [YES]
Logic Crystal       : [YES]
Code Crystal        : [YES]
Speed Crystal       : [YES]
Royal Crystal       : [YES]

ROYAL GATE OPENED
Compilation Successful!
Press any key to continue . . . |
```

Figure :Royal compiler Gate Status

12. Royal Gate Status (Option 10)

- Shows which crystals are collected ([YES]/[NO]).
- If all six crystals are collected → “*ROYAL GATE OPENED*” and compilation is successful.
- Otherwise → “*ROYAL GATE CLOSED*”



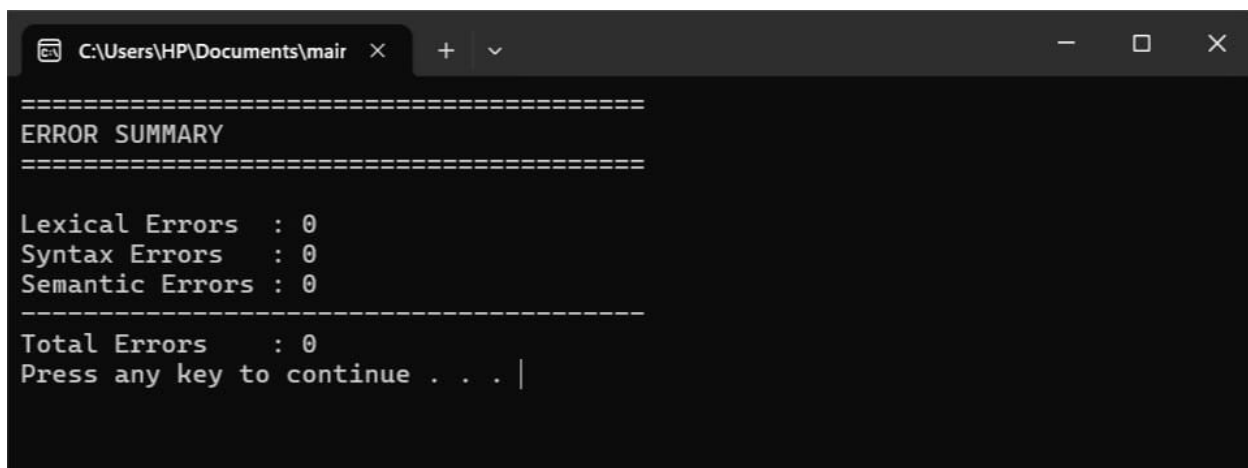
```
C:\Users\HP\Documents\mair x + v
=====
COMPILER STATISTICS
=====

Total Lines : 3
Score       : 100/100
Crystals    : 6 / 6
Press any key to continue . . . |
```

Figure :Compiler Statistics

13. Statistics (Option 11)

- Prints total lines of code, current score (0–100), and number of crystals collected (out of 6).



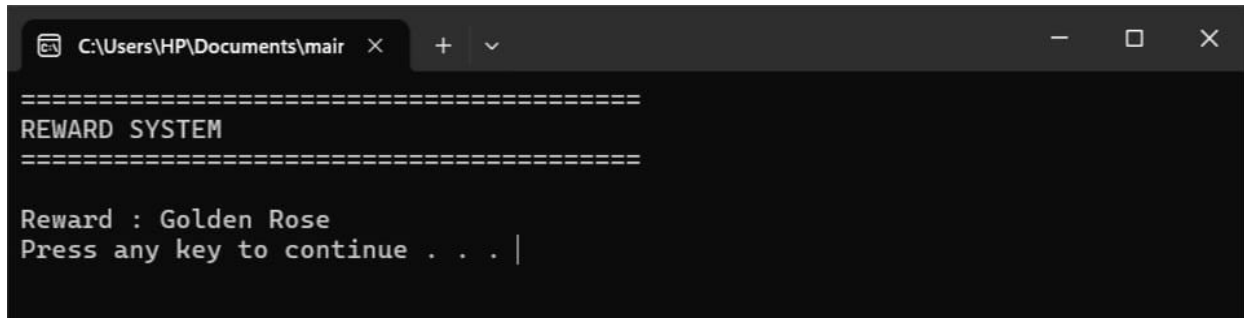
```
C:\Users\HP\Documents\mair x + v
=====
ERROR SUMMARY
=====

Lexical Errors : 0
Syntax Errors  : 0
Semantic Errors : 0
-----
Total Errors   : 0
Press any key to continue . . . |
```

Figure :Error Summary

14. Error Summary (Option 12)

- Lists counts of lexical, syntax, and semantic errors, and the total.



```
=====  
REWARD SYSTEM  
=====
```

Reward : Golden Rose
Press any key to continue . . . |

Figure :Reward system output

15. Reward System (Option 13)

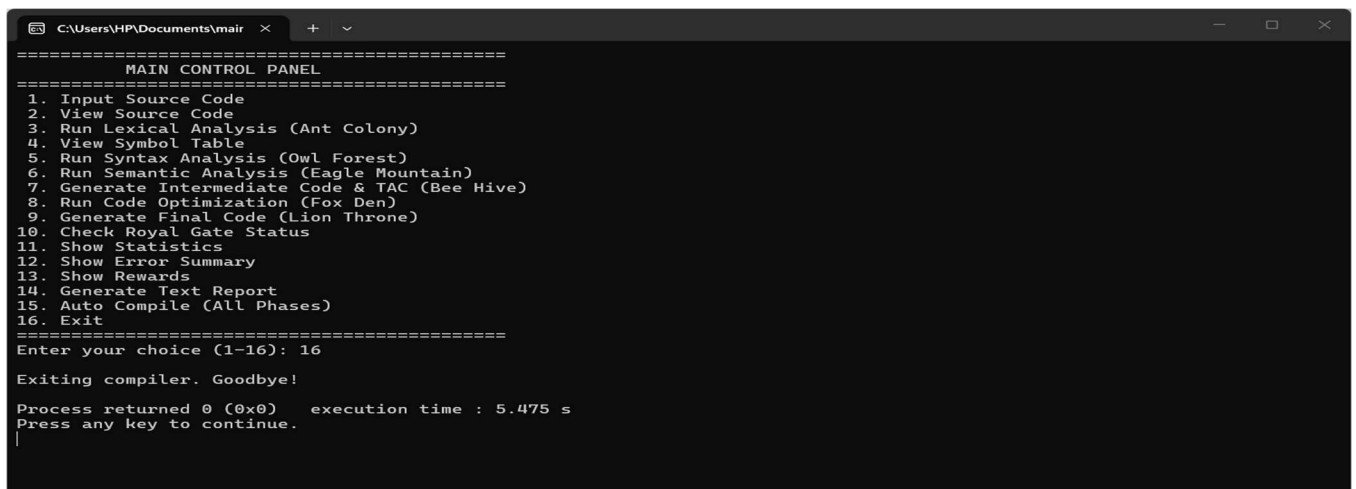
- Gives a reward based on the final score (e.g., Golden Rose for ≥ 90 , Cactus for < 50).

16. Generate Report (Option 14)

- Creates a text file report.txt containing:
 - Source code
 - Symbol table
 - Results of each phase (errors, crystals)
 - TAC, optimized TAC, assembly code
 - Statistics, error counts, and reward.

17. Auto Compile (Option 15)

- Runs all phases automatically (lexical $\rightarrow$ syntax $\rightarrow$ semantic $\rightarrow$ TAC $\rightarrow$ optimization $\rightarrow$ final code).
- Then displays the gate status, statistics, error summary, reward, and generates the report.
- Output: “*Auto Compile Completed!*”.



```
=====  
MAIN CONTROL PANEL  
=====
```

1. Input Source Code
2. View Source Code
3. Run Lexical Analysis (Ant Colony)
4. View Symbol Table
5. Run Syntax Analysis (Owl Forest)
6. Run Semantic Analysis (Eagle Mountain)
7. Generate Intermediate Code & TAC (Bee Hive)
8. Run Code Optimization (Fox Den)
9. Generate Final Code (Lion Throne)
10. Check Royal Gate Status
11. Show Statistics
12. Show Error Summary
13. Show Rewards
14. Generate Text Report
15. Auto Compile (All Phases)
16. Exit

Enter your choice (1-16): 16

Exiting compiler. Goodbye!

Process returned 0 (0x0) execution time : 5.475 s
Press any key to continue.
|

Figure : Exit

18. Exit (Option 16)

- Prints “*Exiting compiler. Goodbye!*” and terminates.

This is a story-themed C++ mini-compiler simulator that runs the full compilation pipeline—lexical, syntax, semantic analysis, TAC generation, optimization, and assembly output—on user-entered source code.

It tracks progress through six “magic crystals” (one per phase), updates a score based on errors, and generates a detailed text report, with all phases accessible individually or via a single auto-compile option.

Future Work:

Future improvements may include:

1. Full C++ syntax support
2. Parser Tree Visualization
3. GUI Version
4. Better Error Recovery
5. More Optimization Techniques
6. Machine Code Generation
7. File Import Feature
8. Syntax Highlighting
9. Cross-platform Support

Limitations:

- ☐ Basic C++ Only: Supports simple variables and statements, but no advanced features like classes or pointers.
- ☐ Simple Error Handling: Catches basic errors without the ability to recover and resume parsing.
- ☐ Pseudo-Assembly: Generates placeholder assembly instructions instead of real machine code.

Conclusion:

The Animal Kingdom Mini C++ Compiler successfully demonstrates the basic phases of compiler design in an interactive and user-friendly way. The project helps students understand compiler concepts such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and final code generation through a simple menu-driven interface.

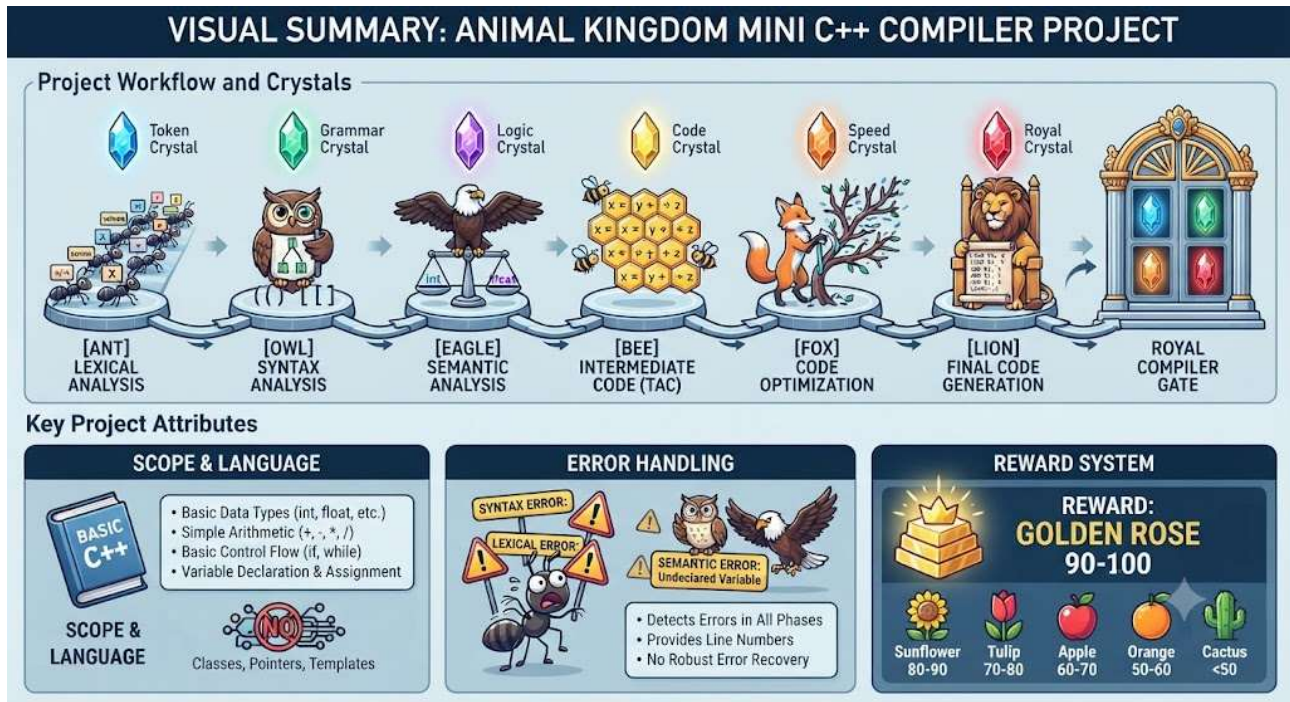


Figure : Final Output (Visualize)

References:

Books

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi & Jeffrey D. Ullman, Compilers: Principles, Techniques and Tools (2nd Edition).
2. Kenneth C. Loudon, Compiler Design.
3. Andrew W. Appel, Modern Compiler Implementation in C.

Online Resources

1. C++ Documentation
2. GCC Compiler Documentation
3. GeeksforGeeks (Compiler Design Articles)
4. TutorialsPoint (Compiler Design)